QML Lecture · 3일차

실습 설명 자료

MediQ-GAN 미세조정 → QGAN과 같은 지표로 비교

오리엔테이션

3일차 진행 순서

노트북 섹션 순서대로 실습하고, 각 블록 앞에 해당 설명 PART를 진행함

§0–1
환경·자산
저장소 clone
checkpoint 로드
PART 0

→

§2
QGAN 기준선
600 step
checkpoint
지표 4종
PART 0

→

§3
MediQ-GAN 구조
dual-stream
16큐빗 × 5
PART 1

→

§4
미세조정
10 epoch에서 출발
+20 step
PART 2

→

§5
비교
같은 지표로
QGAN vs MediQ
PART 3

→

§6
토론
양자 생성모델의
한계·방향
PART 4

운영 포인트 §4 미세조정 셀을 먼저 실행한 뒤 PART 2를 설명함. 진행 로그는 step마다 출력되며 '남은 예상' 시간을 보고 step 수를 줄여도 됨.

PART 0 · 노트북 §1 – §2

2일차 복습과 QGAN 기준선

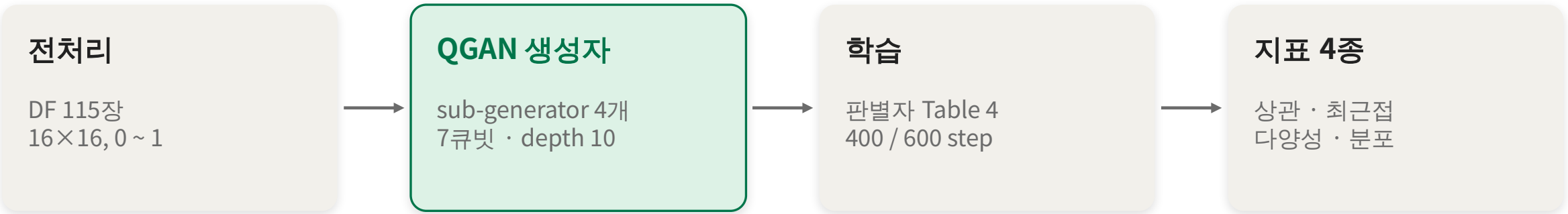
2일차에 만든 QGAN을 checkpoint로 불러와 오늘 비교의 기준선으로 씀

이 파트의 코드	torch.load · PatchQuantumGenerator(**config) · gm.evaluate
핵심 질문	checkpoint에는 무엇이 들어 있는가 / 기준선 숫자를 어떻게 읽는가

PART 0 · 2일차 복습

2일차에 만든 것

전처리 → 양자 생성자 → 학습 → 지표 4종. 오늘은 생성자 구조만 바꾸고 나머지는 그대로 씀



오늘 바뀌는 것과 그대로인 것

	2일차	3일차
생성자	QGAN - 생성자 전체가 양자회로	MediQ-GAN - 고전 stream + 양자 stream
데이터 · 판별자 · 지표	DF 115장 · Table 4 · 지표 4종	동일

checkpoint 불러오기

config로 같은 구조를 먼저 만들고, 학습된 가중치를 덮어씀

```
ckpt_path = f"{ASSET_DIR}/qgan_df_{IMG_SIZE}px.pt"
checkpoint = torch.load(ckpt_path, map_location="cpu")
cfg = checkpoint["config"]

gen = PatchQuantumGenerator(
    n_generators=cfg["n_generators"],
    n_data_qubits=cfg["n_data_qubits"],
    n_ancillas=cfg["n_ancillas"], q_depth=cfg["q_depth"])
gen.load_state_dict(checkpoint["generator_state_dict"])
gen.eval()
```

checkpoint 키	내용
generator_state_dict	생성자 가중치 (회전각 등)
discriminator_state_dict	판별자 가중치 (MediQ-GAN만)
config	모델 재구성에 필요한 설정
history	step별 loss_g · loss_d
epoch	학습한 epoch 수 (MediQ-GAN)

QGAN config sub-generator 4 · 데이터 6 + ancilla 1 · depth 10 · 600 step

QGAN 기준선

강사 checkpoint(600 step)를 지표 4종으로 평가한 값 - 뒤에서 MediQ-GAN과 나란히 놓음

0.195

② 클래스 평균 상관

± 0.012 (64장 × 10시드)

1.64

③ 최근접 실제 거리

0에 가깝지 않음 - 암기 아님

1.92

④ 샘플 간 거리

실제 DF 2.505의 약 77%

```
with torch.no_grad():
    qgan_fake = gen(batch_size=64).numpy().reshape(64, 16, 16)
qgan_scores = gm.evaluate(qgan_fake, df_imgs, "prob")
```

읽는 법 생성 호출마다 새 난수를 쓰므로 실행할 때마다 값이 조금씩 달라짐. 소수점보다 모델 간 격차를 봄

PART 1 · 노트북 §3 직전

MediQ-GAN 구조

생성자 안에서 고전 stream과 양자 stream이 병렬로 돌고 하나로 합쳐짐

이 파트의 코드	ToyDualStreamGenerator · q_circuit · quantum_stream · forward
핵심 질문	'어디까지가 양자인가' / 입력이 어떻게 나뉘고 합쳐지는가 / 양자 부분은 전체의 몇 %인가

hybrid 수준 - 어디까지가 양자인가

QGAN은 생성자 전체가 양자회로, MediQ-GAN은 생성자 내부의 한 갈래만 양자회로

QGAN - model-level hybrid

Andra et al., 2025 · 2일차 실습

- 생성자 = 양자회로 (sub-generator 4개)
- patch를 나눠 만들고 이어붙임
- 출력 = 측정 확률 (patch별 합 0 ~ 1)
- 판별자만 고전

z → [양자회로 × 4] → 이미지

MediQ-GAN - layer-level hybrid

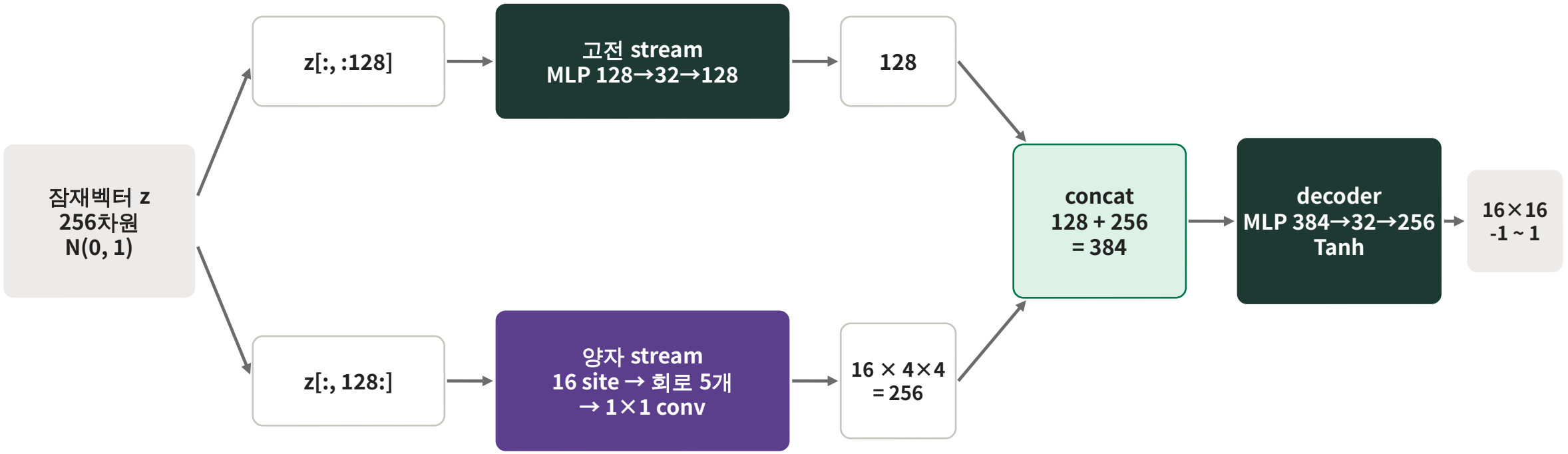
Jiao et al., 2025 · 3일차 실습

- 생성자 = 고전 stream + 양자 stream
- 입력을 절반씩 나눠 병렬 처리 후 concat
- 출력 = 고전 decoder → Tanh (-1 ~ 1)
- 양자 stream은 원 논문 스펙(16큐빗 × 5, depth 8)

z → [고전 ‖ 양자] → concat → decoder

dual-stream의 데이터 흐름

잠재벡터 256차원을 절반씩 나누고, 두 stream의 출력을 이어붙여 decoder로 보냄

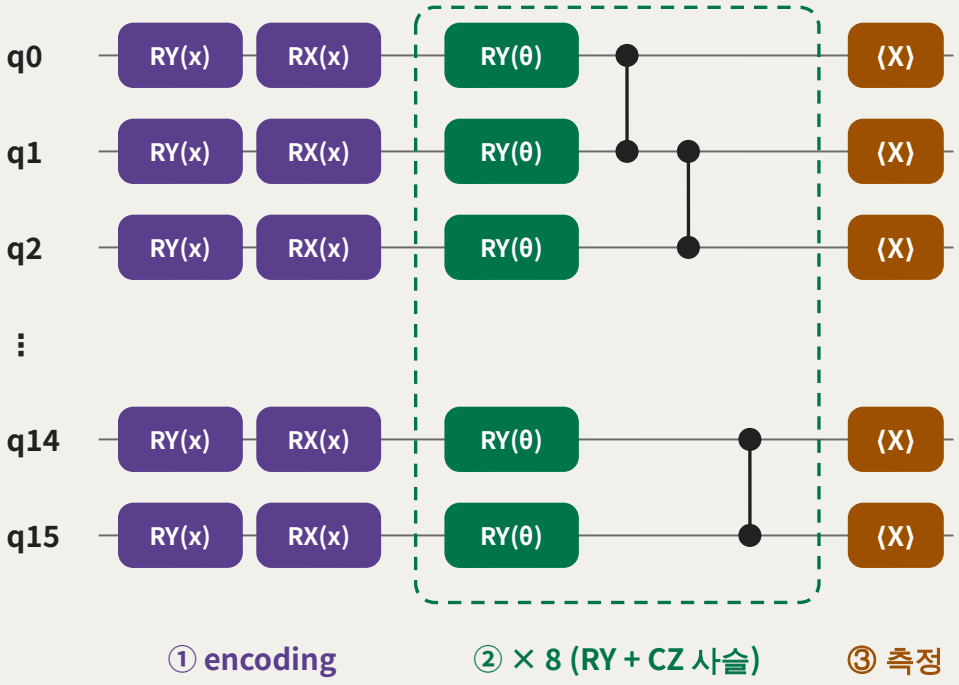



```
z_c, z_q = z[:, :self.half], z[:, self.half:]
fused = torch.cat([self.classical_stream(z_c),
                   self.quantum_stream(z_q)], dim=1)
return self.decoder(fused)
```

양자 회로 - 16큐빗, depth 8

같은 값으로 RY → RX encoding, [학습 RY + 인접 CZ] × 8, 큐빗별 PauliX 기댓값

```
@qml.qnode(dev, interface="torch", diff_method="adjoint")
def q_circuit(x, weights):
    for i in range(n_qubits):          # 1 encoding
        qml.RY(x[i], wires=i)
        qml.RX(x[i], wires=i)
    for layer in range(q_depth):       # 2 x 8
        for i in range(n_qubits):
            qml.RY(weights[layer, i], wires=i)
        for i in range(n_qubits - 1):
            qml.CZ(wires=[i, i + 1])
    return [qml.expval(qml.PauliX(i))  # 3
            for i in range(n_qubits)]
```



출력: 큐빗 16개의 기댓값 → 평균 → site당 스칼라 1개

양자 stream - 16 site 중 5개만 회로로

양자 쪽 입력을 4×4 = 16 site로 보고, 고른 5개 site만 sub-generator 회로가 처리함

site 0 gen 1	site 1 0	site 2 0	site 3 0
site 4 gen 2	site 5 0	site 6 0	site 7 0
site 8 gen 3	site 9 0	site 10 0	**site 11 gen 4**
site 12 0	site 13 0	site 14 0	**site 15 gen 5**

4×4 site 맵 - 보라색 5칸만 회로 출력, 나머지는 0

```
sites  = z_quantum.view(B, 16, self.site_dim)   # (B,16,8)
tokens = self.q_align(sites)                    # (B,16,16)
q_maps = torch.zeros(B, 5, 4, 4)
for g, si in enumerate(self.site_idx):  # [0,4,8,11,15]
    outs = [self.q_circuit(tokens[b, si],
                           self.q_weights[g])
            for b in range(B)]          # B x 16 expvals
    y, x = divmod(si, 4)
    q_maps[:, g, y, x] = outs.mean(dim=1)  # scalar
return self.proj_qexp(q_maps).flatten(1)   # (B, 256)
```

회로 실행 수 = 5 × batch · 16큐빗 상태벡터 65,536차원

PART 1 · 파라미터 분포

양자 부분은 2%, 비용은 대부분

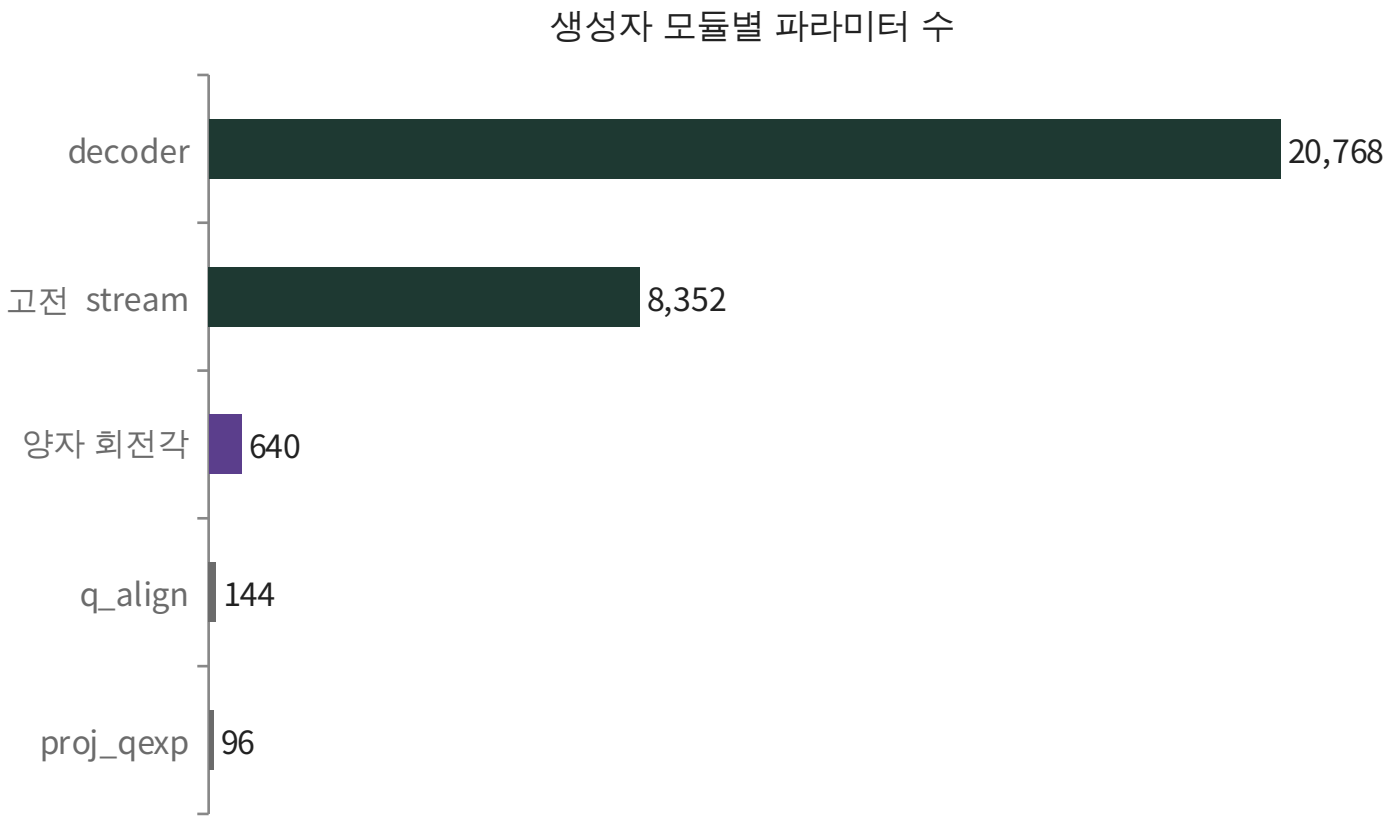
생성자 파라미터 30,000개 중 양자 회전각은 640개(5 × 8층 × 16큐빗)

640

양자 회전각 (2.1%)
5 sub-generator × 8 × 16

30,000

생성자 전체
(2일차 QGAN 생성자 280)



PART 1 · 한계

원 논문과 실습의 차이

양자 stream은 원 논문 스펙 그대로, 고전 부분과 판별자·평가를 단순화함

항목	원 논문 (Jiao et al.)	3일차 실습	비고
양자 stream	16큐빗 × 5, depth 8	**동일**	공식 레포와 같은 회로
고전 encoder / decoder	conv	MLP	16×16 저해상도라 단순화
입력 · 출력	64×64 RGB	16×16 그레이스케일	2일차 전처리 자산 사용
판별자	WGAN-GP critic	표준 BCE 판별자 (Table 4)	QGAN과 같은 판별자
학습	GPU, 전체 학습	25 epoch 사전학습 + 수업 중 미세조정	16큐빗 시뮬레이션 비용
평가	FID, 분류 성능	지표 4종	FID는 16×16에서 의미 없음

PART 2 · 노트북 §4 셀 실행 후

라이브 미세조정

덜 학습된 10 epoch checkpoint에서 출발해 20 step을 더 학습함

이 파트의 코드	mediqgan_e010.pt · sample_z · real_tanh · Adam(betas=(0.5, 0.999))
핵심 질문	왜 전체 학습이 아니라 미세조정인가 / 2일차와 학습 설정이 왜 다른가 / 학습이 불안정한 이유는

PART 2 · 노트북 §4 코드

미세조정 코드 - 2일차와 무엇이 다른가

루프 구조는 2일차와 같고, 척도·잠재벡터·optimizer 설정이 다름

```
mq_ckpt = torch.load(f"{ASSET_DIR}/mediqgan_e010.pt")
mq_disc.load_state_dict(mq_ckpt["discriminator_state_dict"])

real_tanh = torch.tensor(df_imgs.reshape(115, -1) * 2 - 1)
def sample_z(n): return torch.randn(n, 256)   # z ~ N(0, 1)

opt_g = Adam(mq_gen.parameters(),  lr=2e-4, betas=(0.5, 0.999))
opt_d = Adam(mq_disc.parameters(), lr=2e-4, betas=(0.5, 0.999))

for step in range(20):       # batch 4, same D/G loop as day 2
    ...
```

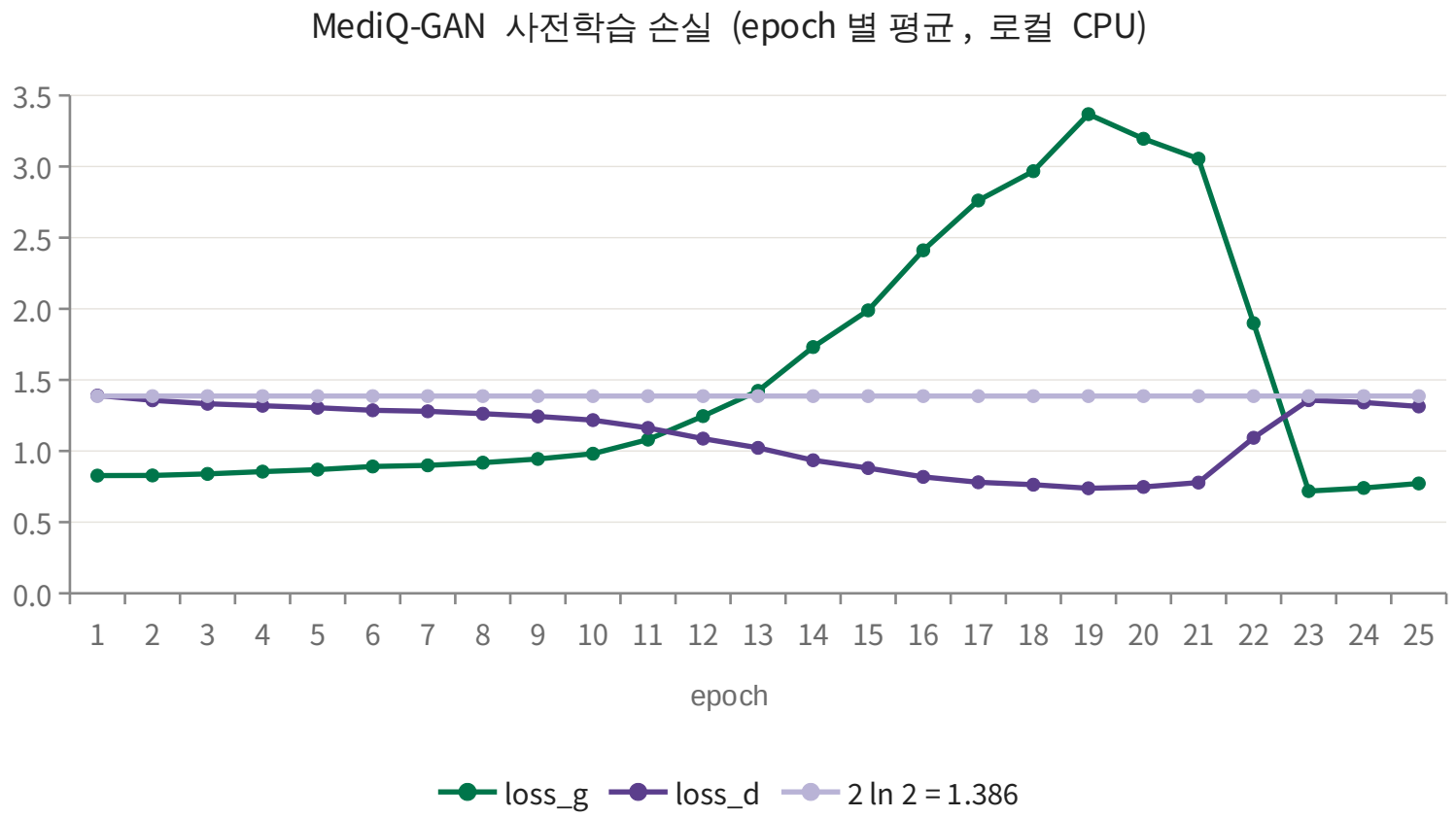
설정	2일차 QGAN	3일차 MediQ
출력 척도	확률 (prob)	Tanh (-1 ~ 1)
실제 이미지	이미지 합 = 1	2x - 1
잠재벡터	U(0, π) 각도	N(0, 1)
학습률 G / D	0.05 / 0.001	2e-4 / 2e-4
Adam betas	(0.9, 0.999)	(0.5, 0.999)
batch · step	8 · 400	4 · 20 (미세조정)

GAN 관례값(DCGAN): lr 2e-4, beta1 0.5

PART 2 · 학습 안정성

25 epoch 동안의 손실 - 판별자가 이기는 구간

강사 checkpoint의 학습 기록(epoch별 평균). 15 ~ 21 epoch에서 판별자가 우세해졌다가 회복함



0.0027

loss_d (lr_g 0.02, 3 epoch)

판별자 완승 - 완전 mode collapse

1.39 → 0.74

loss_d (관례값, 1 → 19 epoch)

판별자 우세 구간 후 22 epoch부터 회복

PART 2 · 계산 비용

왜 전체 학습 대신 미세조정인가

16큐빗 회로 시뮬레이션이 비싸, 25 epoch 사전학습에 196분이 걸림

7.8분

1 epoch (28 step)

checkpoint 기록 평균

196분

25 epoch 사전학습

약 3시간 15분

약 20초

미세조정 1 step

batch 4, 로컬 CPU

	2일차 QGAN	3일차 MediQ-GAN
큐빗 (회로 하나)	7	16
상태벡터 차원	$2^7 = 128$	$2^{16} = 65{,}536$
미분 방식	backprop (default.qubit)	adjoint (lightning.qubit)
수업 중 학습	처음부터 400 step (Colab 약 35분)	10 epoch에서 20 step 미세조정

PART 3 · 노트북 §5 직전

QGAN vs MediQ-GAN

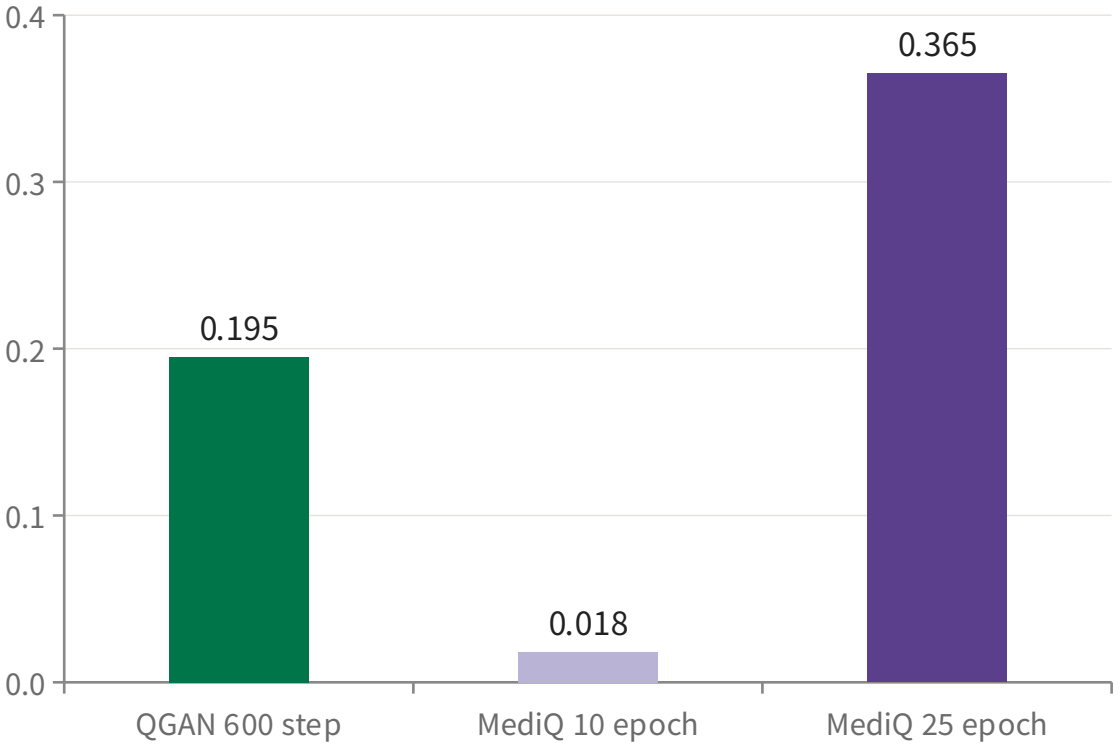
두 모델의 출력을 0~1로 되돌린 뒤 같은 지표 4종으로 비교함

이 파트의 코드	gm.comparison_table · to_unit_scale(x, 'prob' / 'tanh')
핵심 질문	어느 쪽이 더 좋은 생성모델인가 - 형태 재현과 다양성 중 무엇을 볼 것인가

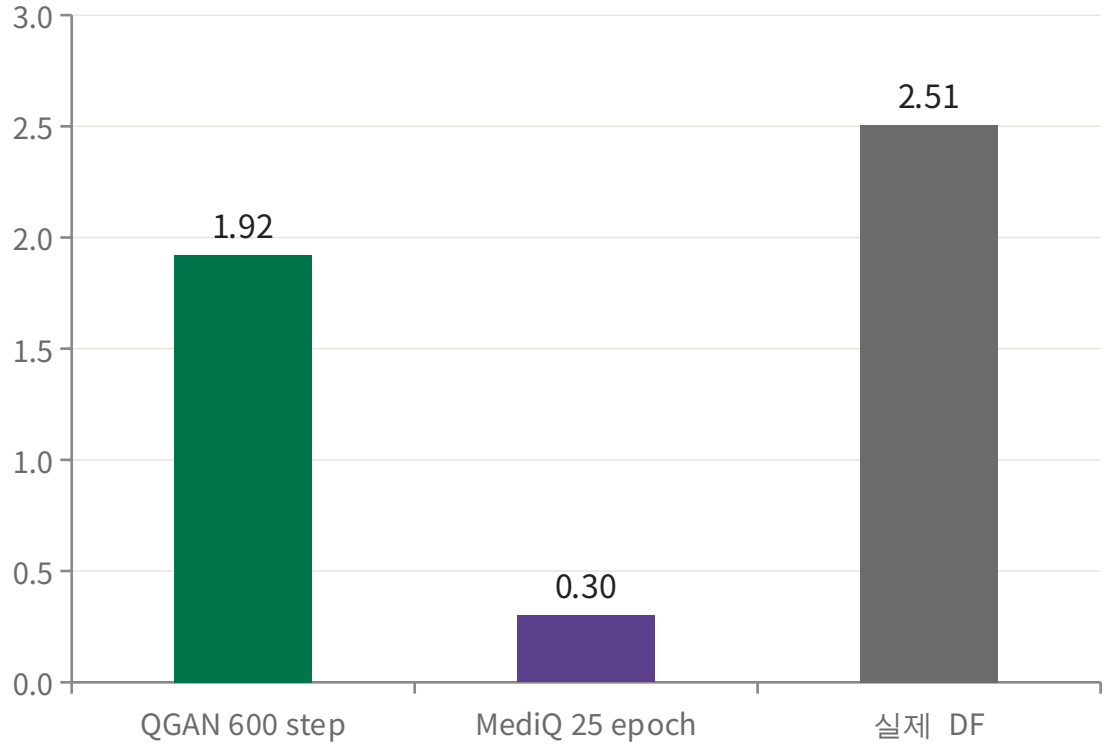
형태는 MediQ-GAN, 다양성은 QGAN

README 결과표 기준. 두 지표를 함께 봐야 '더 좋은 생성모델'에 답할 수 있음

② 클래스 평균 상관 (높을수록 형태 재현)



④ 샘플 간 거리 (0 에 가까우면 mode collapse)



PART 3 · 결과 해석

두 모델이 실패하는 방식이 다름

QGAN은 형태가 약하고, MediQ-GAN은 '평균 이미지'에 가까운 그림을 반복함

지표	QGAN 600 step	MediQ-GAN 25 epoch	읽는 법
② 클래스 평균 상관 ↑	0.195	**0.365**	평균 형태는 MediQ-GAN이 더 가까움
③ 최근접 실제 거리	1.64	3.44	MediQ-GAN은 실제 이미지 어느 것과도 멂
④ 샘플 간 거리 (실제 2.505)	**1.92**	0.30	MediQ-GAN은 비슷한 그림을 반복

QGAN

다양성은 실제에 가깝지만 병변 형태 재현이 약함. 각 patch가 독립적으로 만들어져 이미지 전체 구조를 잡기 어려움

MediQ-GAN

평균 형태는 잘 맞추지만 다양성이 실제의 1/8. 고전 decoder가 출력을 만들어 Tanh 포화 시 z와 무관한 이미지가 나옴

이미지 생성에서의 양자컴퓨팅 - 6가지 관찰

3일 동안의 실습에서 직접 확인한 것

1 **측정 방식이 성패를 가름**

회로가 아니라 측정·해석(post-selection)을 바꾸자 학습이 됨

2 **지금의 학습은 대부분 시뮬레이션**

실제 장치는 parameter-shift - 파라미터당 회로 2회 × shot

3 **해상도 한계 = 평가 한계**

16×16에서는 FID가 무의미. 지표 선택 자체가 연구 과제

4 **큐빗 수에 지수적인 비용**

16큐빗은 12큐빗 대비 상태 16배, 실측 시간 약 20배

5 **고전 쪽 안정화가 결과를 좌우**

BCE 판별자는 쉽게 이김 - 원 논문은 WGAN-GP critic

6 **유효한 영역은 아직 좁음**

양자 데이터·상태 생성이 먼저. 고전 이미지에서의 우위는 검증 단계

PART 4 · 참고

MediQ-GAN 원 논문 결과

논문 수치 인용 - 실습의 16×16 결과와 직접 비교하지 않음

데이터셋	내용	결과
ISIC 2019	피부경검사 8클래스	소수클래스(df, vasc, scc)에서 모든 고전 baseline 대비 최고 또는 차상위 FID
ODIR-5k	안저사진	두 backbone 모두에서 최고 성능
RetinaMNIST	다운샘플 안저사진	5개 클래스 중 4개에서 최저(최고품질) FID

비교 대상

DCGAN, WGAN-GP, StyleGAN2-ADA, FastGAN, diffusion(DiT), 이전 세대 hybrid 모델 MosaiQ

실기기 검증

IBM 양자 장치(ibm_marrakesh)에서 추론 검증

PART 4 · 정리

확인 질문

답은 발표자 노트에 있음

1. QGAN과 MediQ-GAN의 hybrid 수준 차이를 '어디까지가 양자인가'로 설명하면
2. checkpoint에 가중치뿐 아니라 config를 함께 저장하는 이유는
3. MediQ-GAN 양자 회로에 ancilla qubit과 post-selection이 필요 없는 이유는
4. MediQ-GAN의 상관계수가 더 높은데도 더 좋은 생성모델이라고 단정할 수 없는 이유는
5. 학습률을 크게 두거나 BCE 판별자를 쓰면 어떤 문제가 생기고, 원 논문은 어떻게 대응했는가

정리

3일 과정 정리

같은 데이터·같은 지표로 두 hybrid 전략을 나란히 비교함

1일차	데이터셋 · 두 논문 아키텍처 · VQC 학습 방식과 계산 비용
2일차	segmentation 전처리 · sub-generator 회로 · QGAN 학습 · 지표 4종
3일차	MediQ-GAN 구조 · 미세조정 · QGAN과 같은 지표로 비교
결론	형태(상관)와 다양성을 함께 봄 · 양자 비용과 학습 안정화가 결과를 좌우함